2-de

1. • (6) trains the generator to maximize the probability that generated samples are classified as real by the discriminator.
   • (7) trains the discriminator to distinguish real examples from generated ones.
   • If only one were used, the adversarial game would break: using only (6) gives no meaningful discriminator, and using only (7) gives no improvement in generation quality.

2. Ideally, the generator should transform the noise distribution $P(z)$ into the real data distribution $P(X)$.

3. The true equation should be: $\min_G \mathbb{E}_{z \sim P(z)} [\log(1 - P(G(z)))]$

4. With the default settings, the GAN shows a clear improvement over training, the generated images evolve from noise to recognizable handwritten digits after a few hundred iterations. The loss curves are highly oscillatory and do not converge smoothly, which reflects the adversarial nature of training.
   Overall, training is reasonably stable, image quality is good, and diversity is preserved, although some digits appear more frequently than others.

5. • reduce ngf:
   Reducing ngf decreases the capacity of the generator, which leads to slightly blurrier and less detailed digits compared to the default setting. Training remains stable overall, but the loss curves show stronger oscillations. The generated images are still diverse and cover multiple digit classes, but some digits appear more frequently. Overall, reducing ngf improves efficiency but

Scanné avec CamScanner

slightly degrades image quality and balance between the generator and discriminator.

- Change weights initialization

When using the default Pytorch weight initialization instead of the DCGAN-specific initialization, the GAN still learns to generate recognizable digits, but the discriminator tends to dominate more often. While image diversity is preserved, the generated digits are slightly noisier and less sharp. Overall, DCGAN initialization leads to more stable training and better visual quality.

- Replace the training loss of the generation with the "true" loss

Using the true generator loss leads to severe training instability. The discriminator quickly saturates, causing vanishing gradients for the generator. As a result, both generator and discriminator losses collapse to zero, and the generator outputs pure noise instead of meaningfull images.

- Reduce the generator learning rate

Reducing the generator learning rate makes training more stable, with smoother loss evolution and fewer abrupt collapses. The discriminator and generator remain better balanced, which allows the generator to improve more gradually. As a result, the generated digits are clearer and more consistent, while maintaining good diversity, although convergence is slightly slow.

- Reduce the discriminator learning rate

Reducing the discriminator learning rate weakens the discriminator and allows the generator to catch up more easily. Training becomes better balanced, with smoother loss curves and fewer sharp spikes. The generated digits appear clearer and more diverse, and the risk of discriminator dominance is reduced, although training may converge more slowly.

Scanné avec CamScanner

- 30 epochs instead of 5 epochs

With 30 epochs, the images are slightly cleaner, but training remains unstable and losses keep oscillating. The discriminator becomes too strong, which limits further improvement.

The traing time is extremely long, while the gain over 5 epochs is small, so increasing epoch is inefficient here.

- Reduce n_z

Reducing the latent dimension significantly limits the generator's capacity. Training remains relatively seable, but the generator loss increases and fluctuates more, which indicates that the discriminator dominates. The generated digies are still recognizable but image diversity is clearly reduced, with more similar shapes A smaller n_z hurts diversity and expressiveness, even if convergence speed is similar

- Linear interpolation between z_1 and z_2

The interpolation produces smooth and continuous transitions between digits, which shows that the generator has learned a structured latent space.
Digit shapes gradually deform rather than changing abruptly.

- generate 64×64 images

Training becomes significantly more unstable: the discriminator quickly dominates, which leads to clear mode collapse where generated samples are highly similar. Although the discriminator loss is very low, the generator is no longer learning meaningful diversity, which is exacerbated at 64×64 resolution due to increased model capacity, harder optimization and much longer training time.

- Use CIFAR 10

Using CIFAR-10 is much harder than MNIST: training is less seable, losses fluctuate more, and the generated images are blury with weak semantic structure. Although there is some color and texture diversity, the model is clearly underpowered for CIFAR-10 and would require a deeper architecture or longer, more carefully stabilized training.

Scanné avec CamScanner

6. The conditional DCGAN successfully learned to generate digits corresponding to the given labels, with clear class separation and good visual quality. Conditioning helped the generator produce more structured and interpretable outputs compared to the unconditional case, although training was slightly more sensitive to architectural and hyperparameters choices.

7. No. If the discriminator does not receive the label y, it cannot check whether the generated image matches the condition, which breaks the conditional objective. In that case, the generator may ignore the label information and revert to unconditional generation.

8. The training was more successful in terms of semantic control, since the generator learned to associate each label with the correct digit. However, it was slightly harder to stabilize, because the discriminator can more easily overfit using the label information.

9. The noise vector z can be interpreted as controlling style variations within a class, such as stroke thickness, orientation or handwriting style, while the label y determinates the digit identity.

Scanné avec CamScanner